LAPORAN PRATIKUM
STRUKTUR DATA
“ABSTRACT DATA STRUCTURE”

Dosen Pengampu:

DR. Wahyudi. .S.T.M.T

Asisten Lab:

Jovantri Immanuel Gulo

Disusun Oleh:

Marcello Bayu Denar Widjaksono

2511532011

Fakultas Teknologi Informasi

Departemen Informatika

Universitas Andalas

2026

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum mata kuliah Struktur Data, dengan judul: “Abstract Data Structure”.

Pada kesempatan ini, Penulis mengucapkan terima kasih Bapak DR. Wahyudi. .S.T.M.T yang telah memberikan tugas laporan pratikum. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari sempurna dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis mengharapkan segala bentuk saran dan masukan serta kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap Laporan ini dapat memberikan manfaat bagi perkembangan dunia pendidikan.

DAFTAR ISI

KATA PENGANTAR……..i

DAFTAR ISI……ii

BAB I PENDAHULUAN 3

1.1 Latar Belakang 3

1.2 Tujuan..... 3

1.3 Manfaat Pratikum..... 3

BAB II PEMBAHASAN 3

2.1 Pratikum Struktur Data..... **Error! Bookmark not defined.**

2.2 Tugas Praktikum …….....8

BAB III KESIMPULAN……..14

3.1 Kesimpulan ...….……14

DAFTAR PUSTAKA ……...15

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan program komputer, pemahaman mendalam tentang struktur data (data structure) menjadi fondasi utama bagi para programmer. Salah satu bentuk struktur data yang paling sering digunakan adalah Abstract Data Type (ADT) atau Abstract Data Structure. ADT berfungsi untuk mengorganisir data secara efisien sehingga mudah diakses, dimanipulasi, dan dikelola, yang pada akhirnya meningkatkan kinerja serta efisiensi aplikasi yang dibangun.

Pada bahasa pemrograman Java, implementasi ADT dapat dilakukan melalui Java Collections Framework (JCF), yang menyediakan berbagai interface seperti List, Set, dan Map. Kerangka kerja ini memungkinkan pengembang untuk memilih struktur data yang sesuai dengan kebutuhan, seperti ArrayList untuk daftar dinamis atau HashMap untuk pemetaan kunci-nilai. Praktikum ini membahas konsep dasar ADT beserta implementasinya menggunakan JCF, guna memperkuat pemahaman mahasiswa dalam pemrograman berorientasi objek.

1.2 Tujuan Pratikum

1. Menjelaskan konsep Abstract Data Type dalam bahasa pemrograman Java.

1.3 ManfaatPratikum

1. Meningkatkan pemahaman konsep Abstract Data Type dalam bahasa pemrograman Java.
2. Melatih kemampuan mengimplementasikan ADT dalam pengembangan program sederhana.

BAB II
PEMBAHASAN

2.1 Pratikum Striktur Data

```java
package pekan1_2511532011;

public final class Jam_2511532011 {
        private int hh;
        private int mm;
        private int ss;

        public static boolean isValid(int h, int m, int s) {
                return ( 0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
        }

        public Jam_2511532011(int h, int m, int s) {
                this.hh = h ; this.mm = m ;  this.ss = s;
        }

        public int getHour() {return hh;}
        public int getMinute() {return mm;}
        public int  getSecond() {return ss;}

        public void setHour(int h) {this.hh = h;}
        public void setMinutes(int m) {this.mm = m;}
        public void setSecond(int s) {this.ss = s;}

        public int toSeconds () {return hh * 3600 + mm * 60 + ss;}

        public static Jam_2511532011 fromSeconds (int total) {
                if (total < 0 ) throw new IllegalArgumentException("detik negatif");
                total %= 24 * 3600;
                int h = total /3600;
                int m = total /60;
                int s = total % 60;
                return new Jam_2511532011(h,m,s);
        }

        public int compareTo(Jam_2511532011 other) {
                return Integer.compare(this.toSeconds(),other.toSeconds());
        }

        public boolean equals(Object o) {
                if (! (o instanceof Jam_2511532011 j)) return false;
                return hh == j.hh && mm == j.mm && ss == j.ss;
        }

        public int hashCode() {
                return java.util.Objects.hash(hh, mm, ss);
        }

        public Jam_2511532011 plus(Jam_2511532011 other) {
                return fromSeconds(this.toSeconds() + other.toSeconds());
        }

        public Jam_2511532011 minus(Jam_2511532011 other) {
                return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24 *3600 ));
        }


        public Jam_2511532011 nextSeconds() {
                return fromSeconds(this.toSeconds() + 1);
        }

        public Jam_2511532011 nextNSeconds(int n) {
                return fromSeconds(this.toSeconds() + Math.max(0, n));
        }

        public Jam_2511532011 prevSeconds() {
                return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 * 3600));
        }

        public Jam_2511532011 prevNSeconds(int n) {
                return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 * 3600));
```

```
        }

        public static int durasiDetik(Jam_2511532011 jaw, Jam_2511532011 jakh) {
                return jakh.toSeconds() - jaw.toSeconds();
        }

        public String toString() {
                return String.format("%02d:%02d:%02d", hh, mm, ss);
        }


}
```

Penjelasan kode Class Jam_2511532011:

A. Constructor

1. public Jam_2511532011(int h, int m, int s) berfungsi sebagai konstruktor yang memiliki peran untuk menetapkan nilai objek dan dapat dipanggil saat new Jam_2511532011(isi data yang diinputkan sesuai parameter yang telah ditetapkan konstruktor)

B. Getter

1. Public int getHour() berfungsi untuk mengembalikan nilai jam(hh).
2. Public int getMinutes() berfungsi untuk mengembalikan nilai menit(mm).
3. Public int getSecond() berfungsi untuk mengembalikan nilai detik(ss).

C. Setter

1. Public void setHour(int h) berfungsi mengubah nilai jam objek saat ini sesuai dengan input user.
2. Public void setMinutes(int m) berfungsi mengubah nilai menit objek saat ini sesuai dengan input user.
3. Public void setSecond(int s) berfungsi mengubah nilai detik objek saat ini sesuai dengan input user.

D. Operation Method

1. public Jam_2511532011 plus(Jam_2511532011 other) ini berperan untuk menjumlah waktu objek lain dengan waktu objek ini (dijumlah dalam detik dan mengembalikan objek baru hasil penjumlahan).

2. public Jam_2511532011 minus(Jam_2511532011 other) ini berperan untuk mengurangi waktu objek lain dengan waktu objek ini (dikurangi dalam detik dan mengembalikan objek baru hasil pengurangan serta menggunakan Math.floorMod agar hasil negatif tetap benar dalam siklus 24 jam (tidak negative)).

E. Manipulasi waktu

1. public Jam_2511532011 nextSeconds() berperan mendapatkan waktu 1 setelah waktu objek saat ini(objek saat ini + 1).
2. public Jam_2511532011 nextNSeconds(int n) berperan mendapatkan waktu n detik setelahnya (menggunakan Math.max(0, n) agar n nya tidak negative).
3. public Jam_2511532011 prevSeconds() berperan untuk mendapat waktu 1 detik sebelumnya (menggunakan floorMod juga agar siklus waktunya 24 jam).
4. public Jam_2511532011 prevNSeconds(int n) berperan untuk mendapat waktu setelah mundur n detik.

F. Another method/procedural

1. public static boolean isValid(int h, int m, int s) berfungsi untuk mengecek apakah waktunya valid atau tidak, method ini akan mengembalikan nilai true saat kondisi ini terpenuhi:(0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
2. public int toSeconds () berfungsi untuk mengonversi waktu objek saat ini menjadi dalam bentuk detik .
3. public static Jam_2511532011 fromSeconds (int total) ini merupakan method statis yang mengembalikan objek baru berdasarkan input detik.
4. public int compareTo(Jam_2511532011 other) berfungsi untuk membandingkan objek yang digunakan saat ini dengan objek other(sesuai parameter). Serta akan mengembalikan nilai (+/-/=/dll).

5. public boolean equals(Object o) berfungsi untuk mengecek apakah objek saat ini sama dengan objek lain.
6. public int hashCode() ini akan menghasilkan hash code untuk tiap objek (bisa dikatakan juga sebagai kode unik milik tiap objek/ id lah).
7. Public static int durasiDetik()
8. Public String toString() ini berperan untuk menimpa output string bawaan java dengan template output yang telah dibuat.

```
package pekan1_2511532011;

public class JamDriver_2511532011 {

        public static void main(String[] args) {
                Jam_2511532011 a = new Jam_2511532011(23, 59, 50);
                Jam_2511532011 b = new Jam_2511532011(0, 0, 15);

                System.out.println("a     =" + a);
                System.out.println("b     =" + b);
                System.out.println("a+b   =" + a.plus(b));
                System.out.println("next 20s =" + a.nextNSeconds(20));
                System.out.println("durasi (a,b) =" + Jam_2511532011.durasiDetik(a, b));
                System.out.println("a compare to b =" + a.compareTo(b));

        }

}
```

Penjelasan kode class JamDriver:

A. Alur kodenya
   1. Jam_2511532011 a = new Jam_2511532011(23, 59, 50), jadi kita mendeklarasikan variabel (a) dan instansi objeknya dulu.
   2. Jam_2511532011 b = new Jam_2511532011(0, 0, 15), ini juga sama dengan a hanya saja kita membuat objek dengan nama b dengan waktu 15 detik
   3. System.out.println("a =" + a); menampilkan data yang ada di objek a.
   4. System.out.println("b =" + b);.menampilkan data yang ada di objek b.

5. System.out.println("a+b =" + a.plus(b)); ini menjumlahkan waktu objek a dan b.
6. System.out.println("next 20s =" + a.nextNSeconds(20)); menampilkan waktu objek a setelah 20 detik.
7. System.out.println("durasi (a,b) =" + Jam_2511532011.durasiDetik(a, b)); ini menampilkan selisih waktu objek a dan objek b.
8. System.out.println("a compare to b =" + a.compareTo(b)); ini menampilkan hasil perbandiingan waktu objek a dan objek b.

```java
package pekan1_2511532011;
import java.util.Scanner;

public class JamDriver2_2511532011 {

	public static void main(String[] args) {
		Scanner input = new Scanner(System.in);

		System.out.println("=== Program Driver Objek Jam ===");

		System.out.println("\n --- Input Jam 1 ---");
		Jam_2511532011 j1 = buatJamDariInput(input);

		System.out.println("\n --- Input Jam 2 ---");
		Jam_2511532011 j2 = buatJamDariInput(input);

		System.out.println("\n --- Hasil Operasi ---");
		System.out.println("Jam 1 (String)      :" + j1.toString());
		System.out.println("Jam 2 (String)      :" + j2.toString());
		System.out.println("Jam 1 dalam detik   :" + j1.toSeconds());
		System.out.println("Jam 2 dalam detik   :" + j2.toSeconds());

		int perbandingan = j1.compareTo(j2);
		if (perbandingan > 0) {
			System.out.println("Status         : Jam 1 lebih lambat (setelah) jam 2");
		} else if (perbandingan < 0) {
			System.out.println("Status         : Jam 1 lebih awal (sebelum) jam 2");
		} else {
			System.out.println("Status         : Jam 1 dan jam 2 sama persis");
		}

		System.out.println("Durasi (J1 ke J2)         :" + Jam_2511532011.durasiDetik(j1, j2) + "detik");

		Jam_2511532011 jNext = j1.nextSeconds();
		System.out.println("Jam 1 Detik Berikutnya:" + jNext);

		Jam_2511532011 jPrev = j1.prevSeconds();
		System.out.println("Jam 1 Detik Berikutnya:" + jPrev);

		Jam_2511532011 jHasilPlus = j1.plus(j2);
		System.out.println("Hasil J1 + J2     :" + jHasilPlus);

		input.close();
		System.out.println("\n Program Selesai");
	}

	private static Jam_2511532011 buatJamDariInput(Scanner sc) {
		int h, m, s;
		while(true) {
			System.out.print("Masukkan Jam (0-23)   :");
			h = sc.nextInt();

			System.out.print("Masukkan Menit (0-59)   :");
			m = sc.nextInt();

			System.out.print("Masukkan Detik (0-59)   :");
			s = sc.nextInt();

			if (Jam_2511532011.isValid(h, m, s))
				return new Jam_2511532011(h, m, s);
			else {
				System.out.println("input tidak valid");
			}

		}
	}
```

```
}
```

Penjelasan class JamDriver2:

A. Method helper
   1. private static Jam_2511532011 buatJamDariInput(Scanner sc) ini intinya membuat objek jam akan tetapi waktunya diinputkan oleh user (akan di cek valid atau tidaknya)

B. Alur kode
   1. Scanner input = new Scanner(System.in); ini Deklarasi dan instansi objek Scanner dengan panggilan input.
   2. Jam_2511532011 j1 = buatJamDariInput(input); deklarasi objek pertama.
   3. Jam_2511532011 j2 = buatJamDariInput(input); deklarasi objek kedua.
   4. int perbandingan = j1.compareTo(j2); ini menyimpan hasil perbandingan j1 dan j2.
   5. Jam_2511532011 jNext = j1.nextSeconds(); membuat objek baru yang nilai waktunya 1 detik setelah j1.
   6. Jam_2511532011 jPrev = j1.prevSeconds();membuat objek baru yang nilai waktunya 1 detik sebelum j1.
   7. Jam_2511532011 jHasilPlus = j1.plus(j2); ini dijumlah waktu j1 dan j2.
   8. end

2.2 Tugas Pratikum Struktur Data

```java
package pekan1_2511532011;

public class mobil_2511532011 {
    private String nama;
    private String tahun;
    private String cc;
    private String harga;
    private String merek;


    private static final int total = 10;
    private mobil_2511532011[] koleksiMobil = new mobil_2511532011[total];


    private int jumlahMobil = 0;

    public String toString() {
        return "Mobil{" +
                "nama='" + nama + '\'' +
                ", tahun='" + tahun + '\'' +
                ", cc='" + cc + '\'' +
                ", harga='" + harga + '\'' +
                ", merek='" + merek + '\'' +
                '}';
    }

    public mobil_2511532011(String n, String t, String c, String h, String m) {
        this.nama = n;
        this.tahun = t;
        this.cc = c;
        this.harga = h;
        this.merek = m;
    }

    public void infoMobil() {
        if (jumlahMobil == 0) {
            System.out.println("Tidak ada mobil dalam koleksi.");
            return;
        }


        for (int i = 0; i < jumlahMobil; i++) {
            mobil_2511532011 m = koleksiMobil[i];
            System.out.println("mobil :" + m.getN());
            System.out.println("tahun :" + m.getT());
            System.out.println("cc    :" + m.getC());
            System.out.println("harga :" + m.getH());
            System.out.println("merek :" + m.getM());
            System.out.println("------------------------");
        }
    }


    public void setN(String n) { this.nama = n; }
    public void setT(String t) { this.tahun = t; }
    public void setC(String c) { this.cc = c; }
    public void setH(String h) { this.harga = h; }
    public void setM(String m) { this.merek = m; }

    public String getN() { return nama; }
    public String getT() { return tahun; }
    public String getC() { return cc; }
    public String getH() { return harga; }
    public String getM() { return merek; }
```

```java
private mobil_2511532011 inKoleksi(String n) {
    for (int i = 0; i < jumlahMobil; i++) {
        if (koleksiMobil[i].getN().equals(n)) {
            return koleksiMobil[i];
        }
    }
    return null;
}


private int getIndexKoleksi(String n) {
    for (int i = 0; i < jumlahMobil; i++) {
        if (koleksiMobil[i].getN().equals(n)) {
            return i;
        }
    }
    return -1;
}

public void tambahMobil(String n, String t, String c, String h, String m) {

    if (inKoleksi(n) != null) {
        System.out.println("mobil " + n + " ini sudah ada di dalam koleksi");
        return;
    }


    if (jumlahMobil >= total) {
        System.out.println("Koleksi mobil sudah penuh (Maksimal " + total + ")");
        return;
    }

    mobil_2511532011 mobilbaru = new mobil_2511532011(n, t, c, h, m);
    koleksiMobil[jumlahMobil] = mobilbaru;
    jumlahMobil++;

    System.out.println("mobil " + mobilbaru.getN() + " ditambahkan ke dalam koleksi");
}

public void hapusMobil(String n) {
    if (jumlahMobil == 0) {
        System.out.print("anda tidak memiliki mobil");
        return;
    }

    int index = getIndexKoleksi(n);

    if (index != -1) {
        mobil_2511532011 m = koleksiMobil[index];

        for (int i = index; i < jumlahMobil - 1; i++) {
            koleksiMobil[i] = koleksiMobil[i + 1];
        }


        koleksiMobil[jumlahMobil - 1] = null;
        jumlahMobil--;

        System.out.println(" Mobil yang berhasil dihapus" + m);
    } else {
        System.out.println("tidak ada mobil yang dihapus");
    }
}

public void ubahHargaMobil(String n, String h) {
    if (jumlahMobil == 0) {
        System.out.println("tidak ada mobil yang ada untuk diubah");
```

```
            return;
        }

        mobil_2511532011 m = inKoleksi(n);

        if (m != null) {
            m.setH(h);
            System.out.println("Harga mobil " + n + " berhasil diubah menjadi " + h);
        } else {
            System.out.print("tidak ada mobil yang anda cari ");
        }
    }
}
```

Pembahasan Class mobil_2511532011:

A. Getter
   1. Public String getN() berfungsi untuk mengembalikan nilai nama.
   2. public String getT()berfungsi untuk mengembalikan nilai tahun.
   3. public String getC()berfungsi untuk mengembalikan nilai cc.
   4. public String getH()berfungsi untuk mengembalikan nilai harga.
   5. public String getM()berfungsi untuk mengembalikan nilai merek.

B. Setter
   1. public void setN(String n) ini mengubah nilai nama.
   2. public void setT(String n) ini mengubah nilai tahun.
   3. public void setC(String n) ini mengubah nilai cc.
   4. public void setH(String n) ini mengubah nilai harga.
   5. public void setM(String n) ini mengubah nilai merek.

C. Other method
   1. private mobil_2511532011 inKoleksi(String n) berfungsi untuk mencari objek mobil berdasarkan nama, di dalamnya akan dilakukan looping dari index 0 sampai total (KoleksiMobil[i] yang namanya itu cocok dengan yang diinputkan) dan akan mengembalikan objek tersebut
   2. private int getIndexKoleksi(String n) berfungsi untuk mengembalikan index objek (sesuai nama yang diinputkan) dan mekanismenya mirip dengan method inKoleksi.

3. public String toString() ini override method toString bawaan objek.

D. Method dan procedure inti kode ini

1. public void tambahMobil(String n, String t, String c, String h, String m) berfungsi untuk menambahkan mobile ke dalam array koleksiMobile yang mana mekanismenya itu akan melakukan pengecekan apakah sudah ada/tidak di koleksi, jika tidak ada maka akan dibuat dulu objek mobilBaru yang kemudian akan dimasukan ke dalam koleksiBuku[jumlahMobil] dan jumlahMobil++ .
2. public void hapusMobil(String n) berfungsi untuk menghapus mobil berdasarkan namanya. Mekanismenya kita akan mencari posisi mobil yang diingin dihapus dahulu, jika ada dikoleksi maka kita akan menggeser semua elemen di belakang index yang dihapus ke depan untuk menutup celah.(ini dilakukan oleh syntax loop) kemudian kita akan mengosongkan elemen terakhir serta mengurangi kapasitas mobil yang dapat ditampung di koleksiMobil.
3. public void ubahHargaMobil(String n, String h) ini berfungsi untuk mengupdate harga mobil berdasarkan nama mobil yang ingin diubah harganya. Mekanismenya kita akan cek apakah jumlahMobil == 0, jika ada maka kita akan mencari objek mobil berdasarkan nama dan jika objek ditemukan maka kita akan menggunakan setH untuk mengubah harga.

```
package pekan1_2511532011;

public class mobilDriver_2511532011 {

        public static void main(String[] args) {
         mobil_2511532011 dealer = new mobil_2511532011("punya admin hahaha",
"2024", "0", "0", "Admin");

         System.out.println("=== 1. Menambah Mobil ===");
         dealer.tambahMobil("Avanza", "2020", "1300cc", "200 Juta", "Toyota");
         dealer.tambahMobil("Brio", "2021", "1200cc", "180 Juta", "Honda");
         dealer.tambahMobil("Pajero", "2019", "2400cc", "500 Juta",
"Mitsubishi");

         System.out.println("\n=== 2. Coba Tambah Mobil Duplikat ===");
         dealer.tambahMobil("Avanza", "2022", "1500cc", "250 Juta", "Toyota");

         System.out.println("\n=== 3. Lihat Semua Koleksi (Info) ===");
         dealer.infoMobil();

         System.out.println("\n=== 4. Ubah Harga Mobil ===");
         dealer.ubahHargaMobil("Brio", "175 Juta");
         dealer.ubahHargaMobil("Civic", "300 Juta");

         System.out.println("\n=== 5. Lihat Koleksi Setelah Ubah Harga ===");
         dealer.infoMobil();

         System.out.println("\n=== 6. Hapus Mobil ===");
         dealer.hapusMobil("Pajero");
         dealer.hapusMobil("Ferrari");

         System.out.println("\n=== 7. Lihat Koleksi Akhir ===");
         dealer.infoMobil();
    }

}
```

Pembahasan Class mobilDriver_2511532011:

A. Alur kode program

1. mobil_2511532011 dealer = new mobil_2511532011("punya admin hahaha", "2024", "0", "0", "Admin"); kita akan membuat objek baru dengan nama dealer.
2. dealer.tambahMobil("Avanza", "2020", "1300cc", "200 Juta", "Toyota"); kita akan menambahkan mobil ke dalam array koleksi.
3. dealer.tambahMobil("Pajero", "2019", "2400cc", "500 Juta", "Mitsubishi");

4. dealer.tambahMobil("Pajero", "2019", "2400cc", "500 Juta", "Mitsubishi");
5. dealer.infoMobil(); ini digunakan untuk mencetak seluruh isi array koleksiMobil.
6. dealer.ubahHargaMobil("Brio", "175 Juta"); kemudian kita akan mencari mobil dengan nama Brio dan harganya diubah menjadi 175 juta.
7. dealer.ubahHargaMobil("Civic", "300 Juta"); kemudian kita akan mencari mobil dengan nama Civic dan harganya diubah menjadi 300 juta.
8. dealer.infoMobil();
9. dealer.hapusMobil("Pajero") dan kita juga melakukan ini dealer.hapusMobil("Ferrari"); yang mobil ferari tidak ada di dalam array.
10. dealer.infoMobil().

BAB III
KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa struktur data merupakan fondasi dasar yang wajib diketahui oleh programer, di mana Abstract Data Structure (ADS) berperan krusial dalam mengorganisir data agar mudah diakses dan dimanipulasi sehingga berpengaruh signifikan terhadap kinerja dan efisiensi aplikasi. Dalam konteks bahasa pemrograman Java, konsep ini tidak hanya terbatas pada penggunaan Java Collections Framework atau Collection Interface, tetapi juga dapat diimplementasikan secara manual melalui pembuatan kelas custom yang menerapkan prinsip enkapsulasi dan operasi data yang terdefinisi, sebagaimana telah dibuktikan dalam praktikum ini. Dengan demikian, tujuan praktikum untuk menjelaskan konsep serta implementasi Abstract Data Structure telah tercapai, memberikan manfaat berupa pemahaman mendalam dan kemampuan teknis bagi programer untuk merancang struktur data yang optimal sesuai kebutuhan pengembangan perangkat lunak.

DAFTAR PUSTAKA

[1]W3schools,“JavaDataStructur”2026[Daring].Tersediapada:https://www.w3schools.com/java/java_data_structures.asp.[Diakses:2-april-2026]